

AFRILANG Stdlib

std/c/nlptoken

Français. Bibliothèque AFRILANG 'std/c/nlptoken' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : tokelen, tokelupper, tokelower, toketrim, tokesplit, tokeljoin, tokereplace.

```
'import "std/c/nlptoken"' · 'use nlptoken'
```

```
- 'tokelen(s text) → number' - 'tokelupper(s text) → text' - 'tokelower(s text) → text' - 'toketrim(s text) → text' - 'tokesplit(s text, delim text) → list text' - 'tokeljoin(parts list text, delim text) → text' - 'tokereplace(s text, from text, to text) → text'
```

English. AFRILANG library 'std/c/nlptoken' (tier complex, category complex). Exports 7 function(s): tokelen, tokelupper, tokelower, toketrim, tokesplit, tokeljoin, tokereplace.

```
'import "std/c/nlptoken"' · 'use nlptoken'
```

```
- 'tokelen(s text) → number' - 'tokelupper(s text) → text' - 'tokelower(s text) → text' - 'toketrim(s text) → text' - 'tokesplit(s text, delim text) → list text' - 'tokeljoin(parts list text, delim text) → text' - 'tokereplace(s text, from text, to text) → text'
```

Catégorie / Category	Modules complex (c/)
Niveau / Tier	complex
Fonctions / Functions	7

June 16, 2026

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/c/nlptoken' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : tokenLen, tokenUpper, tokenLower, tokenTrim, tokenSplit, tokenJoin, tokenReplace.

'import "std/c/nlptoken"' · 'use nlptoken'

```
- `tokenLen(s text) → number`
- `tokenUpper(s text) → text`
- `tokenLower(s text) → text`
- `tokenTrim(s text) → text`
- `tokenSplit(s text, delim text) → list text`
- `tokenJoin(parts list text, delim text) → text`
- `tokenReplace(s text, from text, to text) → text`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/nlptoken"
use nlptoken
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés – graphes, NLP, réseaux complexes.

3. Référence API

- tokenLen(s text) → number•
- tokenUpper(s text) → text•
- tokenLower(s text) → text•
- tokenTrim(s text) → text•
- tokenSplit(s text, delim text) → list•
- tokenJoin(parts list text, delim text) → text•
- tokenReplace(s text, from text, to text) → text

4. Exemples d'utilisation

```
import "std/c/nlptoken"
use nlptoken
```

```
create result = tokenLen(/* args */)
say result
```

```
create result = tokenUpper(/* args */)
say result
```

```
create result = tokenLower(/* args */)
say result
```

```
create result = tokenTrim(/* args */)
say result
```

```
create result = tokenSplit(/* args */)
say result
```

```
create result = tokenJoin(/* args */)
say result
```

5. Bonnes pratiques

- Préférez ``import "std/c/nlptoken"`` en tête de fichier.
- Lancez ``afriLang check`` avant de livrer.
- Combinez avec les modules stdlib de la même catégorie.
- Voir `/stdlib/` pour le source live et les démos playground.
- Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module nlptoken

```
function tokenLen(s text) returns number
end
```

```
function tokenUpper(s text) returns text
end
```

```
function tokenLower(s text) returns text
end
```

```
function tokenTrim(s text) returns text
end
```

```
function tokenSplit(s text, delim text) returns list text
end
```

```
function tokenJoin(parts list text, delim text) returns text
end
```

```
function tokenReplace(s text, from text, to text) returns text
end
end
```

English

1. Overview

AFRILANG library 'std/c/nlptoken' (tier complex, category complex). Exports 7 function(s): tokenLen, tokenUpper, tokenLower, tokenTrim, tokenSplit, tokenJoin, tokenReplace.

```
'import "std/c/nlptoken"' · 'use nlptoken'
```

```
- `tokenLen(s text) → number`  
- `tokenUpper(s text) → text`  
- `tokenLower(s text) → text`  
- `tokenTrim(s text) → text`  
- `tokenSplit(s text, delim text) → list text`  
- `tokenJoin(parts list text, delim text) → text`  
- `tokenReplace(s text, from text, to text) → text`
```

2. Installation & import

Add the module to your program:

```
import "std/c/nlptoken"  
use nlptoken
```

Tier: complex · Category: Complex modules (c/)
Advanced domains – graphs, NLP, complex networks.

3. API reference

- tokenLen(s text) → number•
tokenUpper(s text) → text•
tokenLower(s text) → text•
tokenTrim(s text) → text•
tokenSplit(s text, delim text) → list•
tokenJoin(parts list text, delim text) → text•
tokenReplace(s text, from text, to text) → text

4. Usage examples

```
import "std/c/nlptoken"  
use nlptoken
```

```
create result = tokenLen(/* args */)   
say result
```

```
create result = tokenUpper(/* args */)   
say result
```

```
create result = tokenLower(/* args */)   
say result
```

```
create result = tokenTrim(/* args */)   
say result
```

```
create result = tokenSplit(/* args */)
say result
```

```
create result = tokenJoin(/* args */)
say result
```

5. Best practices

- Prefer ``import "std/c/nlptoken"`` at the top of your file.
- Run ``afrilang check`` before shipping.
- Combine with related stdlib modules from the same category.
- See `/stdlib/` for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

module nlptoken

```
function tokenLen(s text) returns number
end
```

```
function tokenUpper(s text) returns text
end
```

```
function tokenLower(s text) returns text
end
```

```
function tokenTrim(s text) returns text
end
```

```
function tokenSplit(s text, delim text) returns list text
end
```

```
function tokenJoin(parts list text, delim text) returns text
end
```

```
function tokenReplace(s text, from text, to text) returns text
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/nlptoken"`
- Vérification : `afrilang check`
- Documentation : <https://afrilang.dev/stdlib/std/c/nlptoken/>

Source (excerpt)

```
module nlptoken

function tokenLen(s text) returns number
end

function tokenUpper(s text) returns text
end

function tokenLower(s text) returns text
end

function tokenTrim(s text) returns text
end

function tokenSplit(s text, delim text) returns list text
end

function tokenJoin(parts list text, delim text) returns text
end

function tokenReplace(s text, from text, to text) returns text
end
end
```